

Federated Learning Simulation

محاكاة التعلم الاتحادي

Local training, FedAvg aggregation, non-IID data, and communication analysis

والبيانات غير المتجانسة وتحليل الاتصال FedAvg التدريب المحلي ودمج

د. م. أوس غباش | إعداد: Dr.-Ing. Aouss Gabash

Objectives

- Create multiple smart-grid clients with private local data.
- Train local linear models without sharing raw measurements.
- Implement weighted Federated Averaging.
- Track global loss across communication rounds.
- Compare IID and non-IID client data.
- Study client participation, communication cost, and malicious updates.

الأهداف

- إنشاء عدة عملاء ببيانات محلية خاصة.
- تدريب نماذج محلية دون مشاركة القياسات الخام.
- تنفيذ FedAvg الموزون.
- تتبع خطأ النموذج العام عبر جولات الاتصال.
- مقارنة البيانات المتجانسة وغير المتجانسة.
- دراسة المشاركة الجزئية وكلفة الاتصال والتحديثات الخبيثة.

1. Laboratory Scenario

Five distribution feeders collaboratively train a one-step load-forecasting model. Each client keeps its raw load and temperature data locally and sends only model parameters to the server.

$$\hat{P} = w_0 + w_1 P_{prev} + w_2 T$$

$$w^{t+1} = \sum_k \frac{n_k}{\sum_j n_j} w_k^{t+1}$$

1. سيناريو المختبر

تتعاون خمسة مغذيات لتدريب نموذج تنبؤ بالحمل. تبقى بيانات الحمل والحرارة لدى كل عميل، بينما تُرسل معاملات النموذج فقط إلى الخادم.

2. Complete MATLAB Program

```

clear; clc; close all;
rng(18);

%% Federated-learning settings
K = 5; % Number of clients
rounds = 40; % Communication rounds
localEpochs = 4;
eta = 0.02; % Local learning rate
nSamples = [180 220 260 320 420];

%% Create private client data
Xclient = cell(K,1);
yclient = cell(K,1);

for k = 1:K
    n = nSamples(k);
    temperature = 10 + 20*rand(n,1);
    previousLoad = 40 + 12*randn(n,1) + 3*k;

    % Non-IID behavior: each feeder has a different offset
    clientBias = 2.5*(k-3);
    loadNow = 8 + 0.82*previousLoad + ...
        0.55*temperature + clientBias + 2*randn(n,1);

    Xclient{k} = [ones(n,1), previousLoad, temperature];
    yclient{k} = loadNow;
end

%% Global validation data
Xval = [];
yval = [];
for k = 1:K
    Xval = [Xval; Xclient{k}];
    yval = [yval; yclient{k}];
end

%% Initialize global model
p = size(Xval,2);
wGlobal = zeros(p,1);

```

```

lossHistory = zeros(rounds,1);
clientLoss = zeros(rounds,K);

%% Federated training
for r = 1:rounds
    localModels = zeros(p,K);

    for k = 1:K
        Xk = Xclient{k};
        yk = yclient{k};
        wk = wGlobal;

        for e = 1:localEpochs
            prediction = Xk*wk;
            gradient = (2/size(Xk,1))*Xk'*(prediction-yk);
            wk = wk - eta*gradient;
        end

        localModels(:,k) = wk;
        clientLoss(r,k) = mean((Xk*wk-yk).^2);
    end

    % Sample-weighted FedAvg
    weights = nSamples/sum(nSamples);
    wGlobal = localModels*weights';

    globalPrediction = Xval*wGlobal;
    lossHistory(r) = mean((globalPrediction-yval).^2);
end

%% Results
fprintf('Global model coefficients:\n');
disp(wGlobal);
fprintf('Final global MSE = %.4f\n',lossHistory(end));

figure;
semilogy(1:rounds,lossHistory,'LineWidth',1.8);
grid on;
xlabel('Communication round');
ylabel('Global MSE');

```

```

title('Federated Learning Convergence');

figure;
plot(1:rounds,clientLoss,'LineWidth',1.2);
grid on;
xlabel('Communication round');
ylabel('Local MSE');
title('Per-Client Training Loss');
legend('Client 1','Client 2','Client 3','Client 4','Client 5',...
       'Location','best');

figure;
scatter(yval,Xval*wGlobal,18,'filled'); hold on;
minY = min(yval); maxY = max(yval);
plot([minY maxY],[minY maxY],'--','LineWidth',1.3);
grid on;
xlabel('True load');
ylabel('Predicted load');
title('Global Federated Model');

```

2. البرنامج الكامل

ينشئ البرنامج بيانات خاصة لكل عميل، ثم يدرب نموذجًا خطيًا محليًا في كل جولة، ويجمع النماذج حسب عدد العينات، ويتابع خطأ النموذج العام وأداء كل عميل.

3. How FedAvg Appears in the Code

```

weights = nSamples/sum(nSamples);
wGlobal = localModels*weights';

```

Each column of `localModels` contains one client model. Multiplying by the sample weights produces the global model.

Raw matrices Xclient and yclient never leave their client loops.

3. ظهور FedAvg في الكود

يمثل كل عمود نموذج عميل، ثم تُضرب مصفوفة النماذج بأوزان تتناسب مع عدد العينات للحصول على النموذج العام.

4. Experiment A: Equal vs Sample-Weighted Aggregation

```
% Equal client weights
wEqual = mean(localModels,2);

% Sample-weighted client weights
sampleWeights = nSamples/sum(nSamples);
wWeighted = localModels*sampleWeights';

mseEqual = mean((Xval*wEqual-yval).^2);
mseWeighted = mean((Xval*wWeighted-yval).^2);

fprintf('Equal-weight MSE    = %.4f\n',mseEqual);
fprintf('Sample-weight MSE   = %.4f\n',mseWeighted);
```

Equal weighting treats every feeder equally. Sample weighting gives larger datasets more influence.

4. التجربة A: الدمج المتساوي والموزون

قارن بين منح كل عميل الوزن نفسه ومنحه وزنًا يتناسب مع عدد العينات. راقب أثر ذلك في الخطأ العام وأداء العملاء الصغار.

5. Experiment B: IID Data

```
% Replace different client bias values with one common bias
clientBias = 0;
```

Regenerate all client datasets using the same statistical relationship, then repeat training. Compare convergence speed and final loss with the original non-IID case.

5. التجربة B: البيانات المتجانسة

ألغ الاختلاف بين العملاء وأعد التدريب. غالبًا يصبح التقارب أسرع وأكثر استقرارًا عندما تتشابه توزيعات البيانات.

6. Experiment C: Partial Client Participation

```
participationRate = 0.6;
selectedCount = max(1,round(participationRate*K));
selected = randperm(K,selectedCount);

selectedModels = localModels(:,selected);
selectedSamples = nSamples(selected);
selectedWeights = selectedSamples/sum(selectedSamples);
wGlobal = selectedModels*selectedWeights';
```

Insert this selection step in each round. Run several trials because random participation changes the result.

6. التجربة C: المشاركة الجزئية

اختر نسبة من العملاء في كل جولة بدل مشاركة الجميع. قارن عدد الجولات وكلفة الاتصال واستقرار النتائج.

7. Experiment D: Communication Cost

```
parametersPerModel = numel(wGlobal);
bytesPerParameter = 8;           % double precision
clientsPerRound = K;

uploadBytes = rounds*clientsPerRound*parametersPerModel*bytesPerParameter;
downloadBytes = uploadBytes;
totalMB = (uploadBytes+downloadBytes)/1024^2;

fprintf('Approximate communication = %.4f MB\n',totalMB);
```

$$C \approx 2RMBp$$

The factor two represents downloading and uploading the model.

7. التجربة D: كلفة الاتصال

احسب كمية البيانات المرسلة والمستقبلة اعتمادًا على عدد الجولات والعملاء والمعلومات وحجم كل معلومة.

8. Experiment E: Local Epochs and Client Drift

Repeat training with localEpochs equal to 1, 4, 10, and 25.

Few local epochs	Many local epochs
More communication rounds	Less frequent communication
Smaller local drift	Potentially stronger drift under non-IID data
Higher communication cost	Possible unstable or biased convergence

8. التجربة E: العصور المحلية

زد عدد العصور المحلية وراقب المفاضلة بين تقليل الاتصال وزيادة انجراف النماذج المحلية عندما تختلف البيانات.

9. Experiment F: Malicious Client

```

maliciousClient = 3;
attackScale = 8;
localModels(:,maliciousClient) = ...
    attackScale*localModels(:,maliciousClient);

wMean = localModels*weights';
wMedian = median(localModels,2);

mseMean = mean((Xval*wMean-yval).^2);
mseMedian = mean((Xval*wMedian-yval).^2);

fprintf('FedAvg under attack MSE = %.4f\n',mseMean);
fprintf('Median aggregation MSE = %.4f\n',mseMedian);

```

Median aggregation may improve robustness, but it can also reduce efficiency or reject legitimate unusual clients.

9. التجربة F: عميل خبيث

ضخّم تحديث أحد العملاء ثم قارن FedAvg بالوسيط. لاحظ أن المتانة والأداء والعدالة قد تتعارض.

10. Per-Client Evaluation

```

finalClientMSE = zeros(K,1);
for k = 1:K
    finalClientMSE(k) = mean((Xclient{k}*wGlobal-yclient{k}).^2)
end

disp(table((1:K)',nSamples',finalClientMSE,...
    'VariableNames',{'Client','Samples','MSE'}));

fprintf('Worst-client MSE = %.4f\n',max(finalClientMSE));

```

Global average performance alone can hide weak performance on a small or unusual feeder.

10. تقييم كل عميل

احسب الخطأ لكل عميل وأظهر خطأ أسوأ عميل، لأن المتوسط العام قد يخفي ضعفاً كبيراً لدى بعض المغذيات.

11. Optional Privacy Experiment

```

noiseStd = 0.02;
noisyModels = localModels + noiseStd*randn(size(localModels));
wPrivate = noisyModels*weights';
privacyMSE = mean((Xval*wPrivate-yval).^2);

fprintf('Noisy aggregation MSE = %.4f\n',privacyMSE);

```

This simple noise experiment illustrates the privacy–utility trade-off, but it is not a complete differential-privacy implementation.

11. تجربة الخصوصية الاختيارية

أضف ضوضاء إلى تحديثات العملاء وراقب تراجع الدقة. هذه تجربة توضيحية وليست تطبيقاً كاملاً للخصوصية التفاضلية.

12. Tasks

1. Increase the number of clients from 5 to 20.
2. Use highly unequal dataset sizes.
3. Compare IID and strongly non-IID data.
4. Vary local learning rate and local epochs.
5. Simulate random client dropout.
6. Compare weighted mean, equal mean, median, and trimmed mean.
7. Add a nonlinear feature and compare linear regression with a neural network.
8. Report global MSE, worst-client MSE, rounds, and transmitted bytes.

12. المهام

1. ارفع عدد العملاء إلى 20.
2. استخدم أحجام بيانات مختلفة جداً.
3. قارن البيانات المتجانسة وغير المتجانسة بشدة.
4. غيّر معدل التعلم والعصور المحلية.
5. حاكي انسحاب العملاء عشوائياً.
6. قارن المتوسط الموزون والمتساوي والوسيط والمتوسط المشذب.
7. أضف علاقة غير خطية وقارن الانحدار بشبكة عصبية.
8. اعرض الخطأ العام وخطأ أسوأ عميل والجولات وحجم الاتصال.

13. Mini Project

Build a federated day-ahead load-forecasting system for several buildings or feeders. Include client-specific weather and occupancy patterns, partial participation, communication accounting, one malicious client, and personalized local fine-tuning after global training.

13. المشروع المصغر

ابن نظام تنبؤ اتحاديًا بالأحمال لعدة مبانٍ أو مغذيات، مع اختلاف الطقس والإشغال والمشاركة الجزئية وحساب الاتصال وعميل خبيث وتخصيص النموذج محليًا بعد التدريب العام.